DOCUMENT DE COMMANDE INTERNE

CAHIER DES CHARGES

SmartLogix Prioritizer with IA

Plateforme intelligente de gestion et priorisation
des operations logistiques de derniere mile

Commanditaire :	Direction Operations & Innovation Digitale
Projet :	SmartLogix / SmartTask
Reference document :	CDC-SLX-2026-001
Version :	1.0
Date :	01/09/2026
Classification :	Confidentiel - Usage interne
Destinataire :	Equipe de developpement (prestataire / PFE)
Statut :	Valide pour lancement

Ce document definit les exigences fonctionnelles et non fonctionnelles de la solution a developper. Il sert de reference contractuelle entre le commanditaire et l'equipe de realisation.

Validation du document

Role	Nom	Signature	Date
MOA - Sponsor projet	[A completer]		
MOA - Responsable metier	[A completer]		
MOE - Chef de projet	[A completer]		
MOE - Architecte technique	[A completer]		

Historique des revisions

Version	Date	Auteur	Modifications
0.1	01/2026	Direction SI	Ebauche initiale
0.5	02/2026	Equipe AMOA	Besoins fonctionnels detailles
1.0	01/09/2026	Comite projet	Version validee pour developpement

Table des matieres

1. Introduction et objet du document
2. Contexte et presentation du commanditaire
3. Problematique metier
4. Solution proposee SmartLogix
5. Etude de l'existant
6. Critique de l'existant
7. Besoins fonctionnels
8. Besoins non fonctionnels
9. Contraintes, hypotheses et perimetre
10. Methodologie de realisation adoptee
11. Planning previsionnel et jalons
12. Architecture cible du systeme
13. Modele de donnees et regles metier
14. Interfaces utilisateur et parcours
15. Specifications des API
16. Intelligence artificielle et priorisation
17. Securite et conformite
18. Criteres d'acceptation et recette
19. Livrables attendus
20. Gestion des risques
21. Personas metier et user stories
22. Workflows metier detailles
23. Annexe A - Catalogue complet API
24. Exigences DevOps et deploiement
25. Qualite, tests et communication
26. Glossaire et annexes

CHAPITRE 1 - INTRODUCTION ET OBJET DU DOCUMENT

1.1 Objet

Le present cahier des charges a pour objet de definir, de maniere exhaustive et non ambigue, les exigences auxquelles doit repondre la solution SmartLogix Prioritizer with IA (ci-apres SmartLogix ou la Solution). Ce document est emis par la Direction Operations & Innovation Digitale en qualite de maitre d'ouvrage (MOA) et adresse a l'equipe de developpement chargee de la conception, realisation, tests et mise en production de la plateforme.

1.2 Perimetre du document

Le perimetre couvre l'ensemble du cycle de vie initial du produit : analyse, conception, developpement, integration, tests, documentation utilisateur et deploiement en environnement de pre-production. Sont exclus du present document : la maintenance corrective de niveau 3 apres garantie, l'integration avec des ERP tiers non specifies, et le deploiement multi-pays hors Tunisie.

1.3 Destinataires

Profil	Usage du document
Chef de projet MOE	Pilotage, planning, suivi des exigences
Developpeurs backend/frontend/IA	Specifications techniques detaillees
Architecte	Coherence architecture et integrations
Testeurs / QA	Scenarios de recette et criteres d'acceptation
Product Owner MOA	Validation fonctionnelle
Encadrant academique (PFE)	Evaluation du projet etudiant

1.4 Documents de reference

- Norme ISO/IEC 25010 - Qualite des produits logiciels
- OWASP Top 10 - Securite applicative web
- Guide AMOA logistique derniere mile - secteur transport
- Reglement general sur la protection des donnees (RGPD) - principes applicables
- Documentation interne : processus commande-livraison actuel (v2.3)

CHAPITRE 2 - CONTEXTE ET PRESENTATION DU COMMANDITAIRE

2.1 Presentation de l'organisation

Le commanditaire est une entreprise de logistique et distribution operating in Tunisia, specialisee dans la livraison de colis, documents et produits sensibles (pharmaceutiques, electronique). L'activite couvre le dernier kilometre (last-mile delivery) avec un reseau de chauffeurs independants et salariees, un centre d'appels operateur, et une clientèle B2B/B2C croissante.

2.2 Contexte strategique

Face a la concurrence des plateformes digitales et a l'augmentation des attentes clients (suivi temps reel, fenetres horaires respectees, transparence), la direction a lance un programme de transformation numerique 2025-2027. SmartLogix constitue le pilier operationnel de ce programme : centraliser les commandes, automatiser la priorisation, optimiser les tournées chauffeurs et reduire les echecs de livraison.

2.3 Acteurs metier concernes

Acteur	Role	Volume estime
Administrateur (ADMIN)	Gestion utilisateurs, roles, configuration	2-3
Manager logistique (MANAGER)	Supervision, assignation, IA, analytics	5-10
Operateur (OPERATOR)	Saisie/verification commandes, preparation	15-25
Chauffeur (DRIVER)	Execution livraisons, GPS, preuves	50-150
Client (CLIENT)	Commandes, suivi, avis, incidents	5000+
Systeme IA	Predictions, recommandations, automation	Automatique

2.4 Objectifs strategiques du projet

- Reduire de 25% les retards de livraison dans les 12 mois suivant la mise en production
- Automatiser 60% des commandes prioritaires (HIGH/CRITICAL) sans intervention manuelle
- Ameliorer le taux de reussite au premier passage de livraison de 78% a 90%
- Fournir une visibilite temps reel aux clients finaux
- Industrialiser les decisions d'assignation chauffeur via IA explicable

CHAPITRE 3 - PROBLEMATIQUE METIER

3.1 Constat general

Aujourd'hui, la gestion des operations logistiques repose sur des processus partiellement manualises, des fichiers Excel partages, des echanges telephoniques entre operateurs et chauffeurs, et une absence d'outil unifie de priorisation. Les managers passent en moyenne 2 heures par jour a reconcilier les commandes, identifier les retards potentiels et reassigner manuellement les missions.

3.2 Problemes identifies

3.2.1 Fragmentation des outils

Les commandes sont saisies dans un outil, les taches assignees par email ou WhatsApp, le suivi GPS effectue via une application tierce non integree. Aucune source de verite unique n'existe. Les donnees historiques ne sont pas exploitables pour l'analyse predictive.

3.2.2 Priorisation subjective

La priorisation des livraisons depend de l'experience individuelle du manager. Deux managers peuvent traiter differemment une commande CRITICAL selon leur charge de travail. Il n'existe pas de score objectif combinant distance, historique client, type produit et charge chauffeur.

3.2.3 Retards et echecs de livraison

En 2025, 22% des livraisons ont subi un retard superieur a 30 minutes. 15% des tentatives ont echoue au premier passage (client absent, adresse incorrecte, produit refuse). Ces echecs generent des couts de re-livraison, des reclamations et une degradation de la satisfaction client (NPS actuel : 32).

3.2.4 Manque de visibilite client

Les clients contactent le call center pour connaitre le statut de leur livraison. Aucun portail self-service n'est disponible. Les notifications sont envoiees manuellement par SMS avec un delai moyen de 45 minutes apres changement de statut.

3.2.5 Absence d'intelligence operationnelle

Les decisions d'assignation chauffeur ne tiennent pas compte systematiquement de la performance historique, de la zone geographique, ni du risque de retard predit. L'optimisation d'itineraire est rarement appliquee lorsque un chauffeur a plusieurs missions simultanees.

3.3 Question centrale

Comment concevoir et deployer une plateforme web integrée, securisee et intelligente, permettant de piloter l'ensemble du cycle commande-livraison, de prioriser automatiquement les taches critiques, d'optimiser les tournées chauffeurs et de predire les risques de retard et d'echec de livraison, tout en offrant une experience client temps reel ?

3.4 Enjeux

Enjeu	Impact si non traite
Operationnel	Perte de productivite, surcharge managers
Financier	Couts re-livraison, penalites contractuelles B2B
Client	Churn, avis negatifs, pression concurrentielle
RH	Turnover chauffeurs par manque d'outils
Reglementaire	Tracabilite insuffisante pour produits sensibles

3.5 Impact chiffre estime

logistiques a environ 180 000 TND : re-livraisons (45 000 TND), temps managers (60 000 TND), appels call center suivi (35 000 TND), penalites B2B (25 000 TND), churn client (15 000 TND). SmartLogix vise un ROI positif des la premiere annee si les KPIs cibles sont atteints.

3.6 Parties prenantes impactees

- Direction generale : visibilite KPI strategiques
- Service logistique : outil operationnel quotidien
- Service client : reduction appels suivi
- Chauffeurs : missions clarifiees, moins d'appels
- Clients finaux : experience digitale moderne
- SI interne : maintenance plateforme conteneurisee

CHAPITRE 4 - SOLUTION PROPOSEE SMARTLOGIX

4.1 Vision produit

SmartLogix Prioritizer with IA est une plateforme web multi-acteurs qui couvre le workflow complet : prise de commande, verification, preparation, creation de tache logistique, assignation chauffeur (manuelle ou IA), execution livraison avec preuve, suivi client temps reel, gestion incidents et analytics manager.

4.2 Principes directeurs de la solution

- Architecture microservices leger : backend metier + microservice IA decouple
- Single source of truth : PostgreSQL centralise via Spring Boot
- Securite by design : JWT, RBAC 5 roles, CORS controle
- IA explicable : scores accompagnes de facteurs contributifs
- Automation opt-in : chaine express declenchable automatiquement ou manuellement
- Resilience : fallbacks rule-based si microservice IA indisponible

4.3 Composants majeurs

Composant	Technologie	Port	Responsabilite
Frontend web	Angular 16	4200 / 80	Interfaces multi-role
Backend API	Spring Boot 4.1 / Java 17	9090	Metier, securite, persistence
Microservice IA	Python FastAPI	8000	ML, scoring, chatbot, routes
Base de donnees	PostgreSQL 15+	5432	Donnees transactionnelles
Reverse proxy	Nginx (cible prod)	80/443	Routage, SSL, cache statique

4.4 Flux metier cible

Figure 4.1 - Architecture generale



Le frontend Angular ne communique jamais directement avec le microservice Python. Toutes les requetes IA transitent par Spring Boot qui agit comme orchestrateur, applique les regles metier et journalise les decisions.

4.5 Innovations cles

- Priorisation IA des taches avec score explicable
- Recommandation chauffeur multi-criteres
- Prediction retard et temps de livraison (ML)
- Delivery Success AI : probabilite reussite premier passage
- Automation chaine express 7 etapes pour commandes HIGH/CRITICAL
- Optimisation itineraire TSP pour chauffeurs multi-missions
- Alertes proactives chauffeur (scheduler 5 min)
- Chatbot Copilot contextuel SmartLogix

CHAPITRE 5 - ETUDE DE L'EXISTANT

5.1 Methodologie d'audit

Une mission d'audit AMOA a ete conduite sur 4 semaines (janvier 2026) comprenant : entretiens 12 collaborateurs, shadowing operateurs (3 jours), analyse de 1500 commandes historiques, benchmark 5 solutions du marche, et cartographie AS-IS des processus.

5.2 Processus actuel (AS-IS)

5.2.1 Prise de commande

Le client appelle le centre de relation ou envoie un email. L'operateur saisit la commande dans un tableur partage ou un CRM basique. Validation manuelle par appel retour. Delai moyen de prise en charge : 35 minutes.

5.2.2 Preparation et dispatch

Le manager consulte le tableur chaque matin, cree mentalement un plan de tournées, contacte les chauffeurs par telephone. Aucune trace numerique de la decision d'assignation. Temps moyen d'assignation : 15 minutes par commande urgente.

5.2.3 Execution livraison

Le chauffeur utilise Google Maps independamment. La preuve de livraison est une photo WhatsApp envoyee au manager. Mise a jour statut manuelle dans le tableur en fin de journee.

5.2.4 Suivi client

Le client rappelle le call center. L'operateur contacte le chauffeur par telephone pour obtenir une estimation. Pas de notification automatique.

5.3 Outils existants inventorie

Outil	Usage	Limites
Excel / Google Sheets	Suivi commandes	Pas de workflow, conflits version
WhatsApp	Communication chauffeurs	Non tracable, donnees eparses
CRM basique	Fiche client	Non integre livraison
Google Maps	Navigation	Pas lie aux missions
SMS manuel	Notifications	Lent, couteux, non personnalise

5.4 Benchmark solutions marche

Solution	Points forts	Points faibles vs besoin
Onfleet	Tracking, API	Cout eleve, IA limitee, pas on-premise
Bringg	Orchestration	Complexite, surdimensionne PME
Odoo Logistics	ERP integre	Personnalisation lourde, IA basique
Developpement interne	Adapte metier	Investissement initial

Conclusion audit : aucune solution COTS ne combine priorisation IA explicable, automation chaine express, suivi SSE client et controle total des donnees. Le developpement sur mesure SmartLogix est retenu.

CHAPITRE 6 - CRITIQUE DE L'EXISTANT

6.1 Analyse SWOT de la situation actuelle

Forces

- Equipe metier experimentee (5+ ans en moyenne)
- Base clients fidele malgre les irritants
- Connaissance fine des zones geographiques tunisiennes

Faibless

- Processus non standardises entre equipes
- Donnees non fiables (doublons, statuts obsoletes)
- Dependance aux individus cles (manager senior)
- Zero capacite analytique predictive

Opportu

- Maturite technologique IA/ML accessible (FastAPI, scikit-learn)
- Demande client forte pour suivi digital
- Possibilite d'integrer un projet PFE comme accelerateur

Menaces

- Concurrents 100% digital
- Reglementation tracabilite renforcee (produits sensibles)
- Turnover chauffeurs si outils inadaptes

6.2 Critique detaillee par dimension

6.2.1 Processus

Les processus AS-IS ne possedent pas de workflow formalise avec etats transitions valides. Les statuts sont mis a jour de maniere asynchrone et parfois incoherente (commande DELIVERED alors que tache encore IN_DELIVERY). La Solution DOIT imposer un workflow OrderStatus et TaskStatus avec transitions controlees cote backend.

6.2.2 Donnees

L'absence de base relationnelle centralisee empeche toute requete analytique fiable. Les historiques de performance chauffeur sont estimes, pas mesures. La Solution DOIT persister TaskHistory, AutomationLog, ChatMessage et Review pour audit complet.

6.2.3 Technique

Les outils actuels ne sont pas integrables via API. Aucune authentification centralisee. La Solution DOIT exposer une API REST securisee JWT documentee OpenAPI/Swagger.

6.2.4 Humain

La charge cognitive des managers est excessive. Les operateurs ressaisissent des informations deja connues du CRM. Les chauffeurs perdent du temps a communiquer manuellement. La Solution DOIT reduire les saisies redondantes et automatiser les notifications.

6.3 Ecarts AS-IS vs TO-BE

Capacite	AS-IS	TO-BE SmartLogix
Priorisation	Subjective	Score IA + regles metier
Assignation	Telephone	Recommandation IA + 1 clic
Suivi client	Call center	Portail SSE temps reel
Preuve livraison	WhatsApp	Upload securise + horodatage
Analytics	Inexistant	Dashboard KPI + export CSV

Incidents	Email	Workflow OPEN->RESOLVED

CHAPITRE 7 - BESOINS FONCTIONNELS

7.1 Conventions

Chaque exigence est identifiee par un code BF-XXX (Must = obligatoire, Should = souhaitable, Could = optionnel). Priorite Must sauf indication contraire.

7.2 Authentification et gestion utilisateurs

- [BF-001] (Must) Le systeme doit permettre l'authentification par email/mot de passe avec JWT
- [BF-002] (Must) Le systeme doit supporter 5 roles : ADMIN, MANAGER, OPERATOR, DRIVER, CLIENT
- [BF-003] (Must) L'ADMIN doit gerer le CRUD utilisateurs avec activation/desactivation
- [BF-004] (Must) Chaque utilisateur doit modifier son profil et mot de passe
- [BF-005] (Must) Le systeme doit rediriger vers le dashboard selon le role apres login

7.3 Module Operateur

- [BF-010] (Must) Creer une commande avec client, type produit, priorite, adresse, montant
- [BF-011] (Must) Verifier une commande PENDING -> VERIFIED
- [BF-012] (Must) Preparer une commande READY et creer la tache associee
- [BF-013] (Must) Consulter dashboard KPI : commandes jour, en attente, pretes
- [BF-014] (Must) Preview Delivery Success AI avant enregistrement commande
- [BF-015] (Must) Signaler un incident lie a une commande

7.4 Module Manager

- [BF-020] (Must) Dashboard KPI : taches actives, retards, chauffeurs disponibles
- [BF-021] (Must) CRUD taches avec filtres statut/priorite/recherche
- [BF-022] (Must) Assigner manuellement ou via recommandation IA (assign-recommended)
- [BF-023] (Must) Consulter retards, incidents, analytics avec export CSV
- [BF-024] (Must) Page IA : predict delay, delivery time, delivery success, anomaly
- [BF-025] (Must) Page automation : lancer chaine express, consulter logs
- [BF-026] (Must) Vue cartes et vue liste pour taches et recommandations IA

7.5 Module Chauffeur

- [BF-030] (Must) Consulter missions assignees avec detail adresse/deadline
- [BF-031] (Must) Workflow mission : accept -> start -> delivery -> complete
- [BF-032] (Must) Mettre a jour position GPS (latitude/longitude)
- [BF-033] (Must) Soumettre preuve livraison (image base64)
- [BF-034] (Must) Consulter itineraire optimise IA multi-missions
- [BF-035] (Must) Recevoir alertes proactives retard (scheduler)
- [BF-036] (Must) Consulter historique et performance personnelle

7.6 Module Client

- [BF-040] (Must) Consulter ses commandes et statuts
- [BF-041] (Must) Suivre livraison temps reel via SSE
- [BF-042] (Must) Confirmer reception -> statut RECEIVED
- [BF-043] (Must) Laisser avis note 1-5 avec commentaire (unique par tache)
- [BF-044] (Must) Signaler incident sur commande/livraison

7.7 Intelligence artificielle

- [BF-050] (Must) Microservice Python expose /api/ai/predict/delay (ML ou fallback)
- [BF-051] (Must) Prediction temps livraison /api/ai/predict/delivery-time
- [BF-052] (Must) Delivery Success AI /api/ai/predict/delivery-success
- [BF-053] (Must) Recommandation chauffeur /api/ai/recommend/driver
- [BF-054] (Must) Priorisation tache /api/ai/prioritize/task
- [BF-055] (Must) Optimisation route /api/ai/optimize/route
- [BF-056] (Must) Chatbot Copilot /api/ai/chat avec suggestions
- [BF-057] (Must) Health et model/status pour supervision

7.8 Automation

- [BF-060] (Must) Chaine express 7 etapes : verify, create task, prioritize, assign, notify
- [BF-061] (Must) Declenchement auto sur OrderCreatedEvent si priorite HIGH/CRITICAL
- [BF-062] (Must) Journal AutomationLog avec runId, stepNumber, status

7.9 Cas d'utilisation detailles

CU-01 : S'authentifier

- Acteur : Tous les acteurs
- Preconditions : Compte actif existant
- Declencheur : Soumission formulaire login
- Scenario nominal : Saisir email/mot de passe -> POST /api/auth/login -> recevoir JWT -> redirection dashboard role
- Scenario alternatif : Mot de passe incorrect -> message erreur 401
- Postconditions : Session JWT active 24h

CU-02 : Creer une commande

- Acteur : OPERATOR
- Preconditions : Authentifie OPERATOR, types produit disponibles
- Declencheur : Formulaire nouvelle commande
- Scenario nominal : Selectionner client, type produit ou Autre, priorite -> POST /api/operator/orders -> commande PENDING
- Scenario alternatif : Priorite HIGH -> declenchement automation via OrderCreatedEvent
- Postconditions : Commande enregistree, notification client

CU-03 : Executer chaine automation express

- Acteur : Systeme (automatique) ou MANAGER/OPERATOR (manuel)
- Preconditions : Commande HIGH/CRITICAL, automation enabled
- Declencheur : OrderCreatedEvent ou POST /api/automation/express/{id}
- Scenario nominal : 7 etapes: READ_ORDER, AUTO_VERIFY, CREATE_TASK, AI_PRIORITIZE, AI_ASSIGN_DRIVER, ESTIMATE_AND_NOTIFY, LOG_COMPLETE
- Scenario alternatif : Python IA indisponible -> fallback regles, etape SKIPPED ou FAILED loguee
- Postconditions : Tache creee, journal automation_logs complet

CU-04 : Predire risque de retard

- Acteur : MANAGER
- Preconditions : Tache existante, service IA actif
- Declencheur : Clic analyser sur page Manager IA
- Scenario nominal : POST /api/manager/ai/predict-delay -> WebClient Python -> delay_model.pkl -> probabilite + niveau risque
- Scenario alternatif : Timeout Python -> RuleBasedAIRecommendationService
- Postconditions : Resultat affiche dans interface manager

CU-05 : Optimiser itineraire chauffeur

- Acteur : DRIVER
- Preconditions : Plusieurs missions actives
- Declencheur : Acces page Route chauffeur
- Scenario nominal : GET /api/driver/route/optimized -> Python POST /api/ai/optimize/route -> ordre optimal TSP
- Scenario alternatif : Une seule mission -> ordre trivial
- Postconditions : Liste missions reordonnee affichee

CU-06 : Suivre livraison temps reel

- Acteur : CLIENT
- Preconditions : Commande en livraison
- Declencheur : Acces page suivi livraison
- Scenario nominal : GET /api/client/deliveries/{id}/stream (SSE) -> mises a jour statut/GPS
- Scenario alternatif : Connexion SSE perdue -> reconnexion automatique
- Postconditions : Client informe en temps reel

7.10 Suite cas d'utilisation

CU-07 : Confirmer reception et laisser avis

- Acteur : CLIENT
- Preconditions : Livraison DELIVERED
- Scenario nominal : POST confirm -> POST review rating 1-5 + commentaire
- Postconditions : Commande RECEIVED, avis enregistre

CU-08 : Utiliser chatbot Copilot

- Acteur : Tous (authentifies)
- Preconditions : Session chat initialisee
- Scenario nominal : POST /api/chat/message -> ChatService -> Python /api/ai/chat ou fallback Java
- Postconditions : Message sauve en chat_messages

CU-09 : Gerer utilisateurs

- Acteur : ADMIN
- Preconditions : Role ADMIN
- Scenario nominal : CRUD via /api/users avec validation et BCrypt
- Postconditions : Base users mise a jour

CU-10 : Recevoir alertes proactives

- Acteur : DRIVER
- Preconditions : app.driver.alerts.enabled=true
- Scenario nominal : Analyse missions -> prediction retard -> notification si risque
- Postconditions : Alerte visible dashboard chauffeur

7.11 Matrice acteurs x fonctionnalites

Fonctionnalite	ADM	MGR	OPR	DRV	CLT
Gestion utilisateurs	X				
Dashboard KPI		X	X	X	X
Commandes CRUD			X		R
Taches CRUD		X	R	R	
IA predictions		X	P		
Automation		X	X		
Suivi livraison SSE					X
Chatbot Copilot	X	X	X	X	X

Legende : X = acces complet, R = lecture, P = preview limite

CHAPITRE 8 - BESOINS NON FONCTIONNELS

8.1 Performance

- [BNF-001] (Must) Temps reponse API REST < 500ms pour 95% requetes metier (hors IA)
- [BNF-002] (Must) Prediction IA < 3 secondes timeout configurable
- [BNF-003] (Must) Support 100 utilisateurs concurrents minimum
- [BNF-004] (Should) Chargement initial frontend < 3 secondes

8.2 Disponibilite et fiabilite

- [BNF-010] (Should) Disponibilite cible 99% heures ouvrables
- [BNF-011] (Must) Fallback rule-based si microservice IA down
- [BNF-012] (Must) Migrations Flyway versionnees sans perte donnees

8.3 Securite

- [BNF-020] (Must) Mots de passe hashes BCrypt
- [BNF-021] (Must) JWT expiration configurable (defaut 24h)
- [BNF-022] (Must) RBAC strict sur tous endpoints
- [BNF-023] (Must) CORS restreint aux origines autorisees
- [BNF-024] (Must) Pas de secrets en clair dans le code source

8.4 Utilisabilite

- [BNF-030] (Must) Interface responsive desktop/tablette
- [BNF-031] (Must) Dashboards avec vues cartes et liste
- [BNF-032] (Must) Notifications toast et alertes visuelles
- [BNF-033] (Must) Navigation sidebar par role

8.5 Maintenabilite

- [BNF-040] (Must) Architecture en couches backend (controller/service/repository)
- [BNF-041] (Must) Frontend Angular standalone components
- [BNF-042] (Must) Documentation OpenAPI Swagger backend
- [BNF-043] (Must) Tests unitaires Python (pytest) et Java (JUnit)

8.6 Interoperabilite

- [BNF-050] (Must) API REST JSON UTF-8
- [BNF-051] (Must) Communication backend-IA via HTTP/WebClient
- [BNF-052] (Must) SSE pour streaming suivi client

8.7 Scalabilite et deploiement

- [BNF-060] (Must) Conteneurisation Docker chaque service
- [BNF-061] (Must) Orchestration Docker Compose dev/preprod
- [BNF-062] (Must) CI/CD GitHub Actions build + test
- [BNF-063] (Should) Reverse proxy Nginx production

8.8 Qualite des donnees IA

- [BNF-070] (Must) Dataset documente (synthetique dev, reel prod)
- [BNF-071] (Must) Metriques ML tracees (F1, MAE, R2) dans model_metadata.json
- [BNF-072] (Must) Endpoints ML retournent 503 si modele absent

CHAPITRE 9 - CONTRAINTES, HYPOTHESES ET PERIMETRE

9.1 Contraintes

- Budget projet PFE : ressources humaines limitees (1-2 developpeurs)
- Delai : livraison MVP en 4 mois, version complete en 6 mois
- Stack imposee : Java/Spring, Angular, Python, PostgreSQL
- Hebergement initial : on-premise ou cloud basique (VPS)
- Donnees ML initiales synthetiques faute d'historique exploitable

9.2 Hypotheses

- Les chauffeurs disposent d'un smartphone avec connexion 4G
- PostgreSQL disponible et administre
- Le microservice IA est deploye sur le meme reseau que le backend
- Les utilisateurs sont formes aux roles respectifs

9.3 Perimetre IN

- 5 roles et leurs interfaces dediees
- Workflow commande-tache-livraison complet
- Module IA complet avec fallbacks
- Automation express et alertes chauffeur
- Chatbot Copilot
- Documentation technique et cahier de tests

9.4 Perimetre OUT

- Application mobile native iOS/Android (web responsive suffit MVP)
- Integration ERP/SAP
- Paiement en ligne
- Multi-tenant SaaS
- Mode offline chauffeur

CHAPITRE 10 - METHODOLOGIE DE REALISATION ADOPTEE

10.1 Approche globale : Agile Scrum adapte

La MOA retient une approche Agile Scrum adaptee au contexte PFE : sprints de 2 semaines, backlog priorise par valeur metier, demos fin de sprint au Product Owner, retrospectives pour amelioration continue. Un cycle en V est applique pour les specifications critiques (securite, workflow, donnees) via revues formelles.

10.2 Roles projet

Role Scrum	Responsable	Mission
Product Owner	MOA metier	Prioriser backlog, valider incréments
Scrum Master	Chef de projet MOE	Faciliter, lever impediments
Dev Team	Developpeurs	Concevoir, coder, tester
Architecte	MOE technique	Coherence technique, revues
QA	Testeur	Plans de test, recette

10.3 Phases du projet

Phase	Duree	Livrables
Phase 0 - Cadrage	2 sem	Cahier des charges, backlog initial
Phase 1 - Conception	3 sem	UML, maquettes, architecture, MCD
Phase 2 - Socle technique	2 sem	Auth, users, BDD, CI
Phase 3 - Metier core	4 sem	Commandes, taches, roles
Phase 4 - IA & automation	3 sem	Microservice ML, chaine express
Phase 5 - Frontend avance	3 sem	Dashboards, UX pro, chatbot
Phase 6 - Tests & deploiement	2 sem	Recette, Docker, doc

10.4 Pratiques techniques

- Git flow : branche main protegee, feature branches, PR review
- Definition of Done : code review + tests passants + doc API a jour
- UML : cas utilisation, classes, sequences pour flux critiques
- TDD partiel sur services metier critiques
- Revue architecture avant sprint 3

10.5 Outils de pilotage

- GitHub : versioning, Issues, Projects, Actions CI
- Postman : collection API SmartTask-API
- Swagger UI : documentation interactive backend
- Draw.io / ASCII : diagrammes architecture

CHAPITRE 11 - PLANNING PREVISIONNEL ET JALONS

11.1 Jalons contractuels

Jalon	Date cible	Critere validation
J1 - Kick-off	S0	Equipe constituee, env dev pret
J2 - Socle auth+BDD	S4	Login JWT 5 roles, Flyway OK
J3 - MVP metier	S10	Commande->Tache->Livraison manuel
J4 - IA operationnelle	S14	Predictions + fallback testes
J5 - Beta interne	S18	5 utilisateurs pilotes par role
J6 - Recette MOA	S22	100% Must validés
J7 - Mise en production	S24	Docker Compose prod OK

11.2 Diagramme de Gantt simplifie

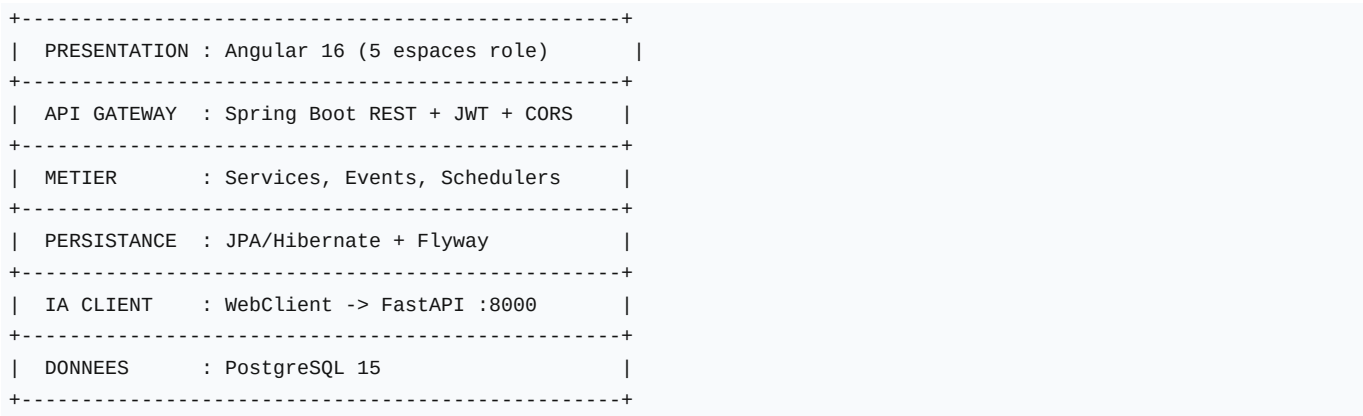
Planning haut niveau (24 semaines)

```
S1-S2   : Cadrage + Conception
S3-S4   : Auth, Users, PostgreSQL, CI
S5-S8   : Orders, Tasks, Operator, Manager
S9-S10  : Driver, Client, SSE tracking
S11-S14 : Python IA, ML training, integration
S15-S16 : Automation express, alertes
S17-S18 : Frontend pro, dashboards, chatbot
S19-S20 : Tests integration, securite
S21-S22 : Recette MOA, corrections
S23-S24 : Docker prod, Nginx, documentation
```

CHAPITRE 12 - ARCHITECTURE CIBLE DU SYSTEME

12.1 Architecture logique

Figure 12.1 - Couches applicatives



12.2 Architecture physique cible

Noeud	Services	Ports
Nginx	Reverse proxy, SSL, static Angular	80, 443
App Server	Spring Boot JAR	9090
AI Server	FastAPI uvicorn	8000
DB Server	PostgreSQL	5432

12.3 Principes d'integration

- Angular -> Spring Boot uniquement (jamais Python direct)
- Spring Boot -> Python via SmartTaskAIClient (WebClient)
- Python -> pas d'acces PostgreSQL (separation concerns)
- Events Spring : OrderCreatedEvent -> AutomationEventListener

CHAPITRE 13 - MODELE DE DONNEES ET REGLES METIER

13.1 Entites principales

User

Attribut	Type	Contrainte
id	Long	PK
firstName, lastName	String	
email	String	unique
password	String	BCrypt
phone	String	
role	Role enum	5 roles
active	boolean	default true
createdAt, updatedAt	LocalDateTime	

DriverProfile

Attribut	Type	Contrainte
id	Long	PK
user	User	OneToOne
status	DriverStatus	AVAILABLE/BUSY/...
zone	String	
currentLoad	int	
totalDeliveries	int	
performanceScore	BigDecimal	

Order

Attribut	Type	Contrainte
id	Long	PK
client, operator	User	ManyToOne
productType	ProductType	ManyToOne
products	String TEXT	
quantity	int	
amount	BigDecimal	
deliveryAddress	String	
priority	Priority	LOW to CRITICAL
status	OrderStatus	workflow
taskId	Long	FK denormalise

Task

Attribut	Type	Contrainte
id	Long	PK
client, driver, createdBy	User	ManyToOne
order	Order	ManyToOne
title, description	String	
status	TaskStatus	workflow
priority	Priority	
deadline	LocalDateTime	
latitude, longitude	BigDecimal	GPS

proofOfDeliveryImage	String	base64
clientConfirmed	boolean	

ProductType

Attribut	Type	Contrainte
id	Long	PK
code	String	unique
name	String	
description	String	
active	boolean	

TaskHistory

Attribut	Type	Contrainte
id	Long	PK
task	Task	ManyToOne
performedBy	User	ManyToOne
action	String	
oldStatus, newStatus	TaskStatus	

13.2 Entites complementaires

Notification

Attribut	Type	Contrainte
id	Long	PK
user	User	ManyToOne
relatedTask	Task	ManyToOne
type	NotificationType	
title, message	String	
readFlag	boolean	

Incident

Attribut	Type	Contrainte
id	Long	PK
reporter	User	ManyToOne
task, order	Task/Order	ManyToOne
type	IncidentType	
status	IncidentStatus	
description	String	

Review

Attribut	Type	Contrainte
id	Long	PK
client	User	ManyToOne
task	Task	OneToOne unique
rating	int	1-5
comment	String	

ChatMessage

Attribut	Type	Contrainte
id	Long	PK
user	User	ManyToOne
role	MessageRole	USER/ASSISTANT
content	String	
sessionId	String	
intent	String	

AutomationLog

Attribut	Type	Contrainte
id	Long	PK
runId	String	UUID execution
chainType	String	EXPRESS
orderId, taskId	Long	
stepNumber	int	1-7
stepName	String	
status	StepStatus	SUCCESS/SKIPPED/FAILED

13.3 Regles metier critiques

- RB-01 : Une commande READY peut generer une tache PREPARED
- RB-02 : Transition tache ASSIGNED -> ACCEPTED par chauffeur uniquement
- RB-03 : Un avis Review est unique par Task (OneToOne)
- RB-04 : Automation express declenche si priorite in (HIGH, CRITICAL)
- RB-05 : Flyway ddl-auto=validate (pas de generation auto schema prod)

Figure 1

```
11 tables PostgreSQL Flyway V1-V10
```

CHAPITRE 14 - INTERFACES UTILISATEUR ET PARCOURS

14.1 Principes UX

- Design professionnel SmartLogix : dashboards, cartes KPI, alertes
- View toggle cartes/liste sur ecrans de gestion
- Toast notifications pour feedback actions
- Alert banners pour nouveautes IA
- Sidebar navigation par role avec logo SL

14.2 Ecrans par role (extrait)

Route	Composant	Acces
/login	LoginComponent	Public
/access-denied	AccessDeniedComponent	Public
/admin/dashboard	AdminDashboardComponent	ADMIN
/admin/users	AdminUsersComponent	ADMIN
/manager/dashboard	ManagerDashboardPageComponent	MANAGER
/manager/tasks	ManagerTasksPageComponent	MANAGER
/manager/planning	ManagerPlanningPageComponent	MANAGER
/manager/map	ManagerMapPageComponent	MANAGER
/manager/drivers	ManagerDriversPageComponent	MANAGER
/manager/delays	ManagerDelaysPageComponent	MANAGER
/manager/incidents	ManagerIncidentsPageComponent	MANAGER
/manager/notifications	ManagerNotificationsPageComponent	MANAGER
/manager/analytics	ManagerAnalyticsPageComponent	MANAGER
/manager/ai	ManagerAiPageComponent	MANAGER
/manager/automation	ManagerAutomationPageComponent	MANAGER
/manager/settings	ManagerSettingsPageComponent	MANAGER
/operator/dashboard	OperatorDashboardPageComponent	OPERATOR
/operator/orders	OperatorOrdersPageComponent	OPERATOR
/operator/orders/create	OperatorOrderCreatePageComponent	OPERATOR
/operator/orders/pending	OperatorPendingPageComponent	OPERATOR

14.3 Parcours critique : commande express

Figure 14.1 - Sequence automation

```
Order HIGH -> Event -> 7 steps automation
```

Parcours utilisateur : Operateur cree commande HIGH -> systeme declenche automation -> Manager recoit notification tache assignee -> Chauffeur voit mission -> Client suit SSE.

CHAPITRE 15 - SPECIFICATIONS DES API

15.1 API Spring Boot (extrait)

Methode	Endpoint	Role	Description
POST	/api/auth/register	PUBLIC	Inscription utilisateur
POST	/api/auth/login	PUBLIC	Authentification JWT
POST	/api/auth/logout	PUBLIC	Deconnexion
GET	/api/auth/me	AUTH	Profil courant
GET	/api/users/me	AUTH	Profil utilisateur
PUT	/api/users/me	AUTH	Modifier profil
PATCH	/api/users/me/password	AUTH	Changer mot de passe
GET	/api/users	ADMIN	Lister utilisateurs
GET	/api/users/{id}	ADMIN	Detail utilisateur
POST	/api/users	ADMIN	Creer utilisateur
PUT	/api/users/{id}	ADMIN	Modifier utilisateur
DELETE	/api/users/{id}	ADMIN	Supprimer utilisateur
PATCH	/api/users/{id}/status	ADMIN	Activer/desactiver
PATCH	/api/users/{id}/role	ADMIN	Changer role
GET	/api/manager/dashboard	MANAGER	KPIs manager
GET	/api/manager/drivers	MANAGER	Liste chauffeurs
GET	/api/manager/drivers/{id}	MANAGER	Detail chauffeur
GET	/api/manager/operators	MANAGER	Liste operateurs
GET	/api/manager/clients	MANAGER	Liste clients
GET	/api/manager/delays	MANAGER	Taches en retard
GET	/api/manager/notifications	MANAGER	Notifications
PATCH	/api/manager/notifications/{id}/read	MANAGER	Marquer lu
GET	/api/manager/analytics	MANAGER	Statistiques
GET	/api/manager/analytics/export	MANAGER	Export CSV
GET	/api/manager/ai/recommendations	MANAGER	Recommandations IA

15.2 Suite endpoints Spring Boot

Methode	Endpoint	Role	Description
GET	/api/manager/ai/recommend-driver/{taskId}	MANAGER	Recommander chauffeur
GET	/api/manager/incidents	MANAGER	Liste incidents
PATCH	/api/manager/incidents/{id}	MANAGER	Traiter incident
GET	/api/manager/ai/health	MANAGER	Sante service IA
GET	/api/manager/ai/model-status	MANAGER	Etat modeles ML
POST	/api/manager/ai/predict-delay	MANAGER	Prediction retard
POST	/api/manager/ai/predict-delivery-time	MANAGER	Prediction ETA
POST	/api/manager/ai/prioritize-task	MANAGER	Prioriser tache
POST	/api/manager/ai/recommend-driver	MANAGER	Recommander chauffeur
POST	/api/manager/ai/detect-anomaly	MANAGER	Detecter anomalie
POST	/api/manager/ai/analyze	MANAGER	Analyse complete IA
GET	/api/tasks	MANAGER	Lister taches
GET	/api/tasks/{id}	MANAGER	Detail tache
GET	/api/tasks/{id}/history	MANAGER	Historique tache

POST	/api/tasks	MANAGER	Creer tache
PUT	/api/tasks/{id}	MANAGER	Modifier tache
DELETE	/api/tasks/{id}	MANAGER	Supprimer tache
PATCH	/api/tasks/{id}/status	MANAGER	Changer statut
PATCH	/api/tasks/{id}/priority	MANAGER	Changer priorite
POST	/api/tasks/{id}/assign/{driverId}	MANAGER	Assigner chauffeur
POST	/api/tasks/{id}/assign-recommended	MANAGER	Assigner recommande
GET	/api/operator/dashboard	OPERATOR	Dashboard operateur
GET	/api/operator/orders	OPERATOR	Lister commandes
GET	/api/operator/orders/pending	OPERATOR	Commandes en attente
GET	/api/operator/orders/{id}	OPERATOR	Detail commande

15.3 API Python IA

Methode	Endpoint	Description
GET	/api/ai/health	Sante service + version
GET	/api/ai/model/status	Etat chargement modeles ML
GET	/api/ai/metrics	Compteurs predictions
POST	/api/ai/predict/delay	Classification risque retard
POST	/api/ai/predict/delivery-time	Regression ETA minutes
POST	/api/ai/detect/anomaly	Detection anomalies statistiques
POST	/api/ai/recommend/driver	Scoring chauffeur rule-based
POST	/api/ai/prioritize/task	Score priorite tache
POST	/api/ai/chat	Reponse chatbot Copilot
GET	/api/ai/chat/suggestions	Suggestions prompts
POST	/api/ai/optimize/route	Optimisation itineraire TSP

15.4 Standards API

- Format JSON, codes HTTP standards (200, 201, 400, 401, 403, 404, 503)
- Erreurs structurees ApiErrorResponse
- Pagination sur listes (page, size, totalElements)
- Documentation Swagger : /swagger-ui.html

CHAPITRE 16 - INTELLIGENCE ARTIFICIELLE ET PRIORISATION

16.1 Objectifs IA

Le module IA doit transformer des donnees operationnelles en decisions actionnables : quelle tache prioriser, quel chauffeur assigner, quel risque de retard anticiper, quelle probabilite de reussite de livraison estimer.

16.2 Modeles ML

Modele	Algorithme	Fichier	Metrique cible
Retard	Logistic Regression	delay_model.pkl	F1 > 0.85
Temps livraison	XGBoost	delivery_time_model.pkl	MAE < 10 min
Reussite livraison	Gradient Boosting	delivery_success_model.pkl	AUC > 0.80
Priorite	Rule-based scoring	N/A	Explicable 100%
Chauffeur	Rule-based scoring	N/A	Explicable 100%

16.3 Pipeline ML

- 1. generate_dataset.py : dataset synthetique documente
- 2. train_*.py : entrainement par modele
- 3. train_all.py : pipeline complet
- 4. ModelService.load_all() au demarrage FastAPI
- 5. Fallback rule-based si .pkl absent ou erreur

16.4 Automation chaine express

Etapes chaine EXPRESS (7 steps)

```
1. READ_ORDER
2. AUTO_VERIFY
3. CREATE_TASK
4. AI_PRIORITIZE
5. AI_ASSIGN_DRIVER
6. ESTIMATE_AND_NOTIFY
7. LOG_COMPLETE
```

16.5 Exigences IA non fonctionnelles

- Latence prediction < 3s
- Explicabilite : facteurs contributifs retournes au frontend
- Versioning modeles via model_metadata.json
- Health check /api/ai/health et /api/ai/model/status

CHAPITRE 17 - SECURITE ET CONFORMITE

17.1 Authentification JWT

Spring Security stateless avec JwtAuthenticationFilter. Token transmis header Authorization Bearer. Secret configurable via JWT_SECRET (min 256 bits prod).

17.2 Matrice autorisation (extrait)

Ressource	ADMIN	MANAGER	OPERATOR	DRIVER	CLIENT
/api/users/**	CRUD	-	-	-	-
/api/manager/**	-	RW	-	-	-
/api/operator/**	-	-	RW	-	-
/api/driver/**	-	-	-	RW	-
/api/client/**	-	-	-	-	RW
/api/tasks/**	-	RW	-	-	-

17.3 Donnees personnelles

- Minimisation : seules donnees necessaires collectees
- Droit acces : profil utilisateur /api/users/me
- Retention : a definir avec DPO (recommandation 3 ans)
- Preuve livraison : stockage securise, acces restreint

17.4 OWASP

- Injection : JPA parametre, validation Jakarta Bean Validation
- Broken Auth : JWT + BCrypt
- XSS : Angular sanitization par defaut
- CSRF : desactive (API stateless JWT)

CHAPITRE 18 - CRITERES D'ACCEPTATION ET RECETTE

18.1 Criteres generaux

- 100% exigences Must implementees et testees
- 0 bug bloquant en recette
- Documentation utilisateur par role livree
- Collection Postman executable

18.2 Scenarios recette (extrait)

ID	Scenario	Resultat attendu
REC-01	Login manager@smarttask.com	Redirect /manager/dashboard
REC-02	Operateur cree commande HIGH	Automation declenchee
REC-03	Manager assign-recommended	Chauffeur assigne, toast OK
REC-04	Chauffeur complete livraison + preuve	Statut DELIVERED
REC-05	Client confirme + avis 5 etoiles	Statut RECEIVED, review OK
REC-06	IA down, predict delay	Fallback rule-based, pas 500
REC-07	Export analytics CSV	Fichier telecharge

18.3 Comptes test exiges

Role	Email	Password
ADMIN	admin@smarttask.com	Admin123!
MANAGER	manager@smarttask.com	Manager123!
OPERATOR	operator1@smarttask.com	Operator123!
DRIVER	driver1@smarttask.com	Driver123!
CLIENT	client1@smarttask.com	Client123!

CHAPITRE 19 - LIVRABLES ATTENDUS

Livrable	Format	Echeance
Code source complet	GitHub repo	S24
Backend Spring Boot	JAR + Dockerfile	S20
Frontend Angular	dist/ + Dockerfile	S20
Microservice IA	Dockerfile existant	S16
Docker Compose	docker-compose.yml	S22
Nginx config	nginx.conf	S23
GitHub Actions CI	.github/workflows/	S8
Documentation API	Swagger + Postman	S20
Cahier de tests	PDF/Markdown	S22
Manuel utilisateur	PDF par role	S24
Rapport PFE / technique	PDF 50 pages	S24

CHAPITRE 20 - GESTION DES RISQUES

Risque	Prob.	Impact	Mitigation
Indisponibilite IA	M	M	Fallback rule-based, health checks
Retard planning PFE	H	H	MVP scope, priorisation Must
Qualite dataset ML	H	M	Dataset synthetique documente, retrain
Performance PostgreSQL	L	M	Index V6 migration, pagination
Fuite secrets Git	M	H	.gitignore, variables env, review
Complexite Angular	M	M	Standalone components, lazy loading
Scope creep	H	H	Perimetre OUT formalise, change control

CHAPITRE 22 - PERSONAS METIER ET SCENARIOS UTILISATEUR

22.1 Persona : Karim - Manager logistique

Karim, 38 ans, responsable de 12 chauffeurs et 4 operateurs. Il arrive a 7h30, consulte les commandes urgentes et assigne les missions. Il perd 2h/jour sur Excel et WhatsApp. Il veut un dashboard unique, des alertes retard automatiques et une recommandation IA fiable pour l'assignation.

22.2 Persona : Sami - Operateur

Sami, 26 ans, traite 40-60 appels/jour. Il saisit les commandes, verifie les coordonnees client et prepare les colis. Il a besoin d'un formulaire rapide, d'une preview IA reussite livraison avant validation, et d'un suivi clair des commandes en attente.

22.3 Persona : Mohamed - Chauffeur

Mohamed, 32 ans, effectue 15-20 livraisons/jour en zone Tunis. Il utilise son smartphone pour naviguer. Il souhaite voir ses missions triees par priorite, un itineraire optimise, des alertes si risque retard, et un upload simple de preuve livraison.

22.4 Persona : Ahmed - Client

Ahmed, 45 ans, commande des pieces auto et medicaments. Il exige transparence : ou est mon colis, quand arrivera-t-il ? Il veut un portail self-service, notifications temps reel et possibilite de confirmer reception et noter le service.

22.5 User stories prioritaires (Product Backlog)

ID	Role	Story	Priorite	Points
US-01	Manager	Voir KPIs jour en 1 ecran	Must	5
US-02	Operateur	Creer commande en < 2 min	Must	8
US-03	Manager	Assigner chauffeur recommande IA	Must	13
US-04	Chauffeur	Accepter et demarrer mission mobile	Must	8
US-05	Client	Suivre livraison temps reel	Must	13
US-06	Manager	Predire risque retard tache	Must	8
US-07	Operateur	Preview score reussite livraison	Must	5
US-08	Systeme	Automation express HIGH/CRITICAL	Must	13
US-09	Chauffeur	Itineraire optimise multi-stops	Should	8
US-10	Tous	Chatbot Copilot contextuel	Should	5
US-11	Manager	Export analytics CSV	Should	3
US-12	Admin	Gerer utilisateurs et roles	Must	5

CHAPITRE 23 - WORKFLOWS METIER DETAILLES

23.1 Cycle de vie OrderStatus

Statut	Description	Acteur declencheur	Statut suivant
PENDING	Commande saisie, non verifiee	OPERATOR create	VERIFIED
VERIFIED	Informations validees	OPERATOR verify	PREPARING/READY
PREPARING	En cours preparation	OPERATOR	READY
READY	Pret pour dispatch	OPERATOR prepare	IN_DELIVERY
IN_DELIVERY	En cours livraison	DRIVER start	DELIVERED
DELIVERED	Livre chez client	DRIVER complete	RECEIVED
RECEIVED	Confirme par client	CLIENT confirm	Final
CANCELLED	Annulee	MANAGER/OPERATOR	Final

23.2 Cycle de vie TaskStatus

Statut	Description	Transition
PREPARED	Tache creee, non assignee	-> ASSIGNED
ASSIGNED	Chauffeur designe	-> ACCEPTED (driver)
ACCEPTED	Chauffeur a accepte	-> IN_DELIVERY
IN_DELIVERY	Livraison en cours	-> DELIVERED
DELIVERED	Livraison effectuee	-> COMPLETED
COMPLETED	Cycle termine	Final
CANCELLED	Annulee	Final

23.3 Diagramme cas d'utilisation global

Figure 23.1 - Cas d'utilisation

```
Voir Figure 5.1 du rapport
```

23.4 Diagramme de classes metier

Figure 23.2 - Classes

```
User-Task-Order-ProductType relations (voir Chapitre 6)
```

23.5 Sequence authentification

Figure 23.3 - Auth JWT

```
Login -> JWT -> Bearer requests
```

CHAPITRE 24 - ANNEXE A - CATALOGUE COMPLET DES API

24.1 Endpoints Spring Boot (suite 50-75)

Methode	Endpoint	Role	Description
POST	/api/operator/orders	OPERATOR	Creer commande
PUT	/api/operator/orders/{id}	OPERATOR	Modifier commande
POST	/api/operator/orders/{id}/verify	OPERATOR	Verifier commande
GET	/api/operator/tasks	OPERATOR	Taches operateur
POST	/api/operator/tasks/{orderId}/prepare	OPERATOR	Preparer commande
GET	/api/operator/clients	OPERATOR	Clients
GET	/api/operator/incidents	OPERATOR	Incidents
POST	/api/operator/incidents	OPERATOR	Signaler incident
GET	/api/driver/dashboard	DRIVER	Dashboard chauffeur
GET	/api/driver/tasks	DRIVER	Missions assignees
GET	/api/driver/tasks/{id}	DRIVER	Detail mission
PATCH	/api/driver/tasks/{id}/accept	DRIVER	Accepter mission
PATCH	/api/driver/tasks/{id}/start	DRIVER	Demarrer livraison
PATCH	/api/driver/tasks/{id}/delivery	DRIVER	En livraison
PATCH	/api/driver/tasks/{id}/complete	DRIVER	Terminer livraison
PATCH	/api/driver/tasks/{id}/location	DRIVER	Mettre a jour GPS
POST	/api/driver/tasks/{id}/proof	DRIVER	Preuve livraison
GET	/api/driver/route	DRIVER	Itineraire missions
GET	/api/driver/route/optimized	DRIVER	Itineraire optimise IA
POST	/api/driver/alerts/check	DRIVER	Verifier alertes IA
POST	/api/driver/incidents	DRIVER	Signaler incident
GET	/api/driver/history	DRIVER	Historique livraisons
GET	/api/driver/performance	DRIVER	Performance chauffeur
GET	/api/client/dashboard	CLIENT	Dashboard client
GET	/api/client/orders	CLIENT	Mes commandes

24.2 Endpoints Spring Boot (suite 75-100)

Methode	Endpoint	Role	Description
GET	/api/client/orders/{id}	CLIENT	Detail commande
GET	/api/client/deliveries/{id}	CLIENT	Suivi livraison
GET	/api/client/deliveries/{id}/stream	CLIENT	SSE temps reel
POST	/api/client/deliveries/{id}/confirm	CLIENT	Confirmer reception
POST	/api/client/incidents	CLIENT	Signaler incident
POST	/api/client/reviews	CLIENT	Laisser avis
GET	/api/client/reviews	CLIENT	Mes avis
GET	/api/client/notifications	CLIENT	Notifications
GET	/api/automation/status	MGR/OP	Statut automation
POST	/api/automation/express/{orderId}	MGR/OP	Lancer chaine express
GET	/api/automation/logs	MANAGER	Journal automation
GET	/api/automation/logs/run/{runId}	MGR/OP	Logs par execution
GET	/api/product-types	AUTH	Types produit
GET	/api/chat/init	AUTH	Initialiser chatbot

POST	/api/chat/message	AUTH	Envoyer message chat
GET	/api/chat/history	AUTH	Historique chat

24.3 Routes Angular completes

Route	Composant	Acces
/login	LoginComponent	Public
/access-denied	AccessDeniedComponent	Public
/admin/dashboard	AdminDashboardComponent	ADMIN
/admin/users	AdminUsersComponent	ADMIN
/manager/dashboard	ManagerDashboardPageComponent	MANAGER
/manager/tasks	ManagerTasksPageComponent	MANAGER
/manager/planning	ManagerPlanningPageComponent	MANAGER
/manager/map	ManagerMapPageComponent	MANAGER
/manager/drivers	ManagerDriversPageComponent	MANAGER
/manager/delays	ManagerDelaysPageComponent	MANAGER
/manager/incidents	ManagerIncidentsPageComponent	MANAGER
/manager/notifications	ManagerNotificationsPageComponent	MANAGER
/manager/analytics	ManagerAnalyticsPageComponent	MANAGER
/manager/ai	ManagerAiPageComponent	MANAGER
/manager/automation	ManagerAutomationPageComponent	MANAGER
/manager/settings	ManagerSettingsPageComponent	MANAGER
/operator/dashboard	OperatorDashboardPageComponent	OPERATOR
/operator/orders	OperatorOrdersPageComponent	OPERATOR
/operator/orders/create	OperatorOrderCreatePageComponent	OPERATOR
/operator/orders/pending	OperatorPendingPageComponent	OPERATOR
/operator/tasks	OperatorTasksPageComponent	OPERATOR
/operator/tasks/tracking	OperatorTrackingPageComponent	OPERATOR
/operator/incidents	OperatorIncidentsPageComponent	OPERATOR
/operator/settings	OperatorSettingsPageComponent	OPERATOR
/driver/dashboard	DriverDashboardPageComponent	DRIVER
/driver/tasks	DriverTasksPageComponent	DRIVER
/driver/tasks/:id	DriverTaskDetailPageComponent	DRIVER
/driver/route	DriverRoutePageComponent	DRIVER
/driver/incidents	DriverIncidentsPageComponent	DRIVER
/driver/history	DriverHistoryPageComponent	DRIVER
/driver/performance	DriverPerformancePageComponent	DRIVER
/driver/settings	DriverSettingsPageComponent	DRIVER
/client/dashboard	ClientDashboardPageComponent	CLIENT
/client/orders	ClientOrdersPageComponent	CLIENT
/client/orders/:id	ClientOrderDetailPageComponent	CLIENT
/client/delivery/:id	ClientDeliveryPageComponent	CLIENT
/client/notifications	ClientNotificationsPageComponent	CLIENT
/client/reviews	ClientReviewsPageComponent	CLIENT
/client/incidents	ClientIncidentsPageComponent	CLIENT
/client/settings	ClientSettingsPageComponent	CLIENT
/profile	ProfileComponent	AUTH

CHAPITRE 25 - EXIGENCES DEVOPS ET DEPLOIEMENT

25.1 Conteneurisation

- [DEV-001] (Must) Dockerfile multi-stage pour backend Spring Boot (Java 17)
- [DEV-002] (Must) Dockerfile multi-stage pour frontend Angular (nginx alpine)
- [DEV-003] (Must) Dockerfile Python existant smarttask-ai/Dockerfile valide
- [DEV-004] (Must) docker-compose.yml orchestrant postgres, backend, ai, frontend, nginx

25.2 Variables d'environnement

Variable	Service	Description	Exemple
DB_URL	Backend	JDBC PostgreSQL	jdbc:postgresql://db:5432/smarttask_db
DB_USERNAME	Backend	User BDD	postgres
DB_PASSWORD	Backend	Mot de passe BDD	[secret]
JWT_SECRET	Backend	Cle JWT 256+ bits	[secret]
AI_SERVICE_URL	Backend	URL microservice IA	http://ai:8000
CORS_ORIGINS	Backend	Origines autorisees	http://localhost
SPRING_BOOT_URL	Python	URL backend pour CORS	http://backend:9090
MODEL_PATH	Python	Repertoire modeles ML	./app/models

25.3 Nginx reverse proxy

- Route / -> frontend Angular static
- Route /api/ -> proxy backend :9090
- Route /swagger-ui/ -> proxy backend (dev/preprod)
- Compression gzip, cache assets statiques
- Headers securite : X-Frame-Options, X-Content-Type-Options

25.4 GitHub Actions CI

- Workflow backend : ./gradlew test sur push/PR
- Workflow frontend : npm ci && npm run build
- Workflow Python : pip install && pytest
- Declenchement sur branches main et develop

25.5 Environnements

Env	Usage	BDD	IA
dev	Developpement local	PostgreSQL local	uvicorn reload
docker	Integration conteneurs	postgres service	ai service
preprod	Recette MOA	PostgreSQL dedie	Modeles ML entraines
prod	Production	PostgreSQL HA	Modeles ML + monitoring

CHAPITRE 26 - QUALITE, TESTS ET COMMUNICATION

26.1 Strategie de tests

Niveau	Outil	Couverture cible	Responsable
Unitaire Java	JUnit 5 + Spring Test	Services metier > 60%	Dev backend
Unitaire Python	pytest	API + services > 70%	Dev IA
Unitaire Angular	Jasmine/Karma	Services core	Dev frontend
Integration	Postman/Newman	Endpoints critiques 100%	QA
E2E manuel	Scenarios recette	Parcours Must 100%	MOA + QA
Securite	OWASP checklist	Top 10 verifie	Architecte

26.2 Plan de communication projet

- Reunion kick-off : presentation cahier des charges a l'equipe
- Daily standup 15 min (equipe dev)
- Demo sprint : fin de chaque sprint 2 semaines
- Comite pilotage : mensuel MOA/MOE
- Rapport avancement : hebdomadaire email sponsor

26.3 Plan de formation utilisateurs

Cible	Duree	Contenu	Support
Operateurs	2h	Commandes, verification, incidents	Guide PDF + video
Managers	3h	Dashboard, IA, automation, analytics	Atelier pratique
Chauffeurs	1h	Missions, GPS, preuve, route IA	Fiche memo mobile
Clients	N/A	Portail self-service intuitif	FAQ en ligne

26.4 Gestion des changements

Toute demande hors perimetre Must doit faire l'objet d'une fiche de changement evaluee par le Product Owner (impact planning, cout, risque). Les exigences Should peuvent etre reportees en phase 2 sans remettre en cause la recette MVP.

CHAPITRE 27 - INDICATEURS DE PERFORMANCE ET BUDGET

27.1 KPIs de succes projet

KPI	Baseline AS-IS	Cible TO-BE	Mesure
Taux retard livraison	22%	< 12%	Analytics mensuel
Echec 1er passage	15%	< 10%	Delivery Success AI + reviews
Temps assignation	15 min/cmd	< 2 min	Automation logs
NPS client	32	> 50	Enquete trimestrielle
Productivite manager	2h/j admin	< 30 min/j	Time tracking
Disponibilite plateforme	N/A	> 99%	Monitoring

27.2 Budget previsionnel (estimation)

Poste	Estimation	Commentaire
Developpement PFE (6 mois)	Ressource humaine	1-2 etudiants + encadrant
Infrastructure VPS/Cloud	50-100 EUR/mois	4 conteneurs + PostgreSQL
Nom de domaine + SSL	30 EUR/an	Let's Encrypt gratuit
Outils (GitHub, Postman)	Gratuit	Plans community
Formation utilisateurs	Interne MOA	2 jours ateliers
Maintenance an 1	A definir	20% effort dev initial

27.3 Indicateurs de suivi projet (EVM simplifie)

Le chef de projet suivra mensuellement : pourcentage exigences Must implementees, velocity moyenne (story points/sprint), nombre bugs ouverts vs fermes, couverture tests, et respect jalons J1-J7. Un tableau de bord GitHub Projects sera partage avec le sponsor.

CHAPITRE 28 - GESTION INCIDENTS ET NOTIFICATIONS

28.1 Types d'incidents

Type	Description	Acteurs signaleurs
DELIVERY_FAILED	Echec livraison	DRIVER, CLIENT
DAMAGED_PRODUCT	Produit endommage	CLIENT, DRIVER
WRONG_ADDRESS	Adresse incorrecte	DRIVER, OPERATOR
CLIENT_ABSENT	Client absent	DRIVER
VEHICLE_ISSUE	Probleme vehicule	DRIVER
OTHER	Autre	Tous

28.2 Workflow incident

Cycle OPEN -> IN_PROGRESS -> RESOLVED -> CLOSED. Le manager traite via /api/manager/incidents/{id}. Chaque incident est lie a une commande et/ou tache. Notification automatique au manager lors de creation.

28.3 Systeme de notifications

- NotificationType : TASK_ASSIGNED, DELAY_RISK, DELIVERY_UPDATE, INCIDENT, SYSTEM
- Notifications persistees en base avec readFlag
- Affichage dashboard par role (manager, client)
- Alertes proactives chauffeur via DriverProactiveAlertScheduler

28.4 Migrations Flyway

Version	Fichier	Contenu
V1	create_users_table	Users, roles, auth
V2	create_tasks_and_logistics	Tasks, drivers, history
V3	orders_incidents_reviews	Orders, incidents, reviews
V4	add_order_received_status	Statut RECEIVED
V5	seed_client_confirm	Donnees seed confirmation
V6	performance_indexes	Index performance
V7	proof_of_delivery	Preuve livraison image
V8	chat_messages	Historique chatbot
V9	automation_logs	Journal automation IA
V10	product_types	Catalogue types produit

CHAPITRE 29 - SYNTHESE EXECUTIVE POUR LA DIRECTION

29.1 Resume en une page

SmartLogix Prioritizer with IA repond a une problematique operationnelle critique : fragmentation des outils, priorisation subjective, retards frequents et absence de visibilite client. La solution propose une plateforme web integrée Angular + Spring Boot + Python IA + PostgreSQL, avec automation chaine express, predictions ML et dashboards professionnels multi-role.

Investissement estime : 6 mois de developpement, infrastructure cloud modeste, retour attendu via reduction retards (-25%), amelioration NPS et productivite managers. Le present cahier des charges constitue le contrat fonctionnel entre la MOA et l'equipe de realisation.

29.2 Decision attendue

- Valider le perimetre Must et les jalons J1-J7
- Nommer le Product Owner MOA et le chef de projet MOE
- Allouer les ressources infrastructure (PostgreSQL, VPS/Docker)
- Autoriser le demarrage Sprint 1

29.3 Contacts projet

Role	Organisation	Contact
Sponsor	Direction Operations	[A completer]
Product Owner	Service Logistique	[A completer]
Chef de projet MOE	Equipe developpement	[A completer]
Architecte SI	Direction SI	[A completer]
Encadrant PFE	Etablissement	[A completer]

CHAPITRE 21 - GLOSSAIRE ET ANNEXES

21.1 Glossaire

Terme	Definition
SmartLogix	Nom commercial de la plateforme
SmartTask	Nom technique du code source
Last-mile	Dernier kilometre de livraison
JWT	JSON Web Token pour authentification
SSE	Server-Sent Events pour streaming temps reel
TSP	Travelling Salesman Problem - optimisation route
Flyway	Outil migrations base de donnees
RBAC	Role-Based Access Control
MOA	Maitrise d'ouvrage (commanditaire)
MOE	Maitrise d'oeuvre (realisation)
MVP	Minimum Viable Product
CRUD	Create Read Update Delete

21.2 Annexes

- Annexe A : Liste complete endpoints (cf. rapport_data.py)
- Annexe B : Migrations Flyway V1 a V10
- Annexe C : Diagrammes UML (dossier docs/diagrammes/)
- Annexe D : Collection Postman SmartTask-API

21.3 Approbation finale

La signature des parties mentionnees en page de validation vaut acceptation du present cahier des charges et autorise le demarrage de la phase de realisation de SmartLogix Prioritizer with IA.